

Engenheiro de IA, especialista na plataforma Claude

Compartilhado

+ Criar notebook

Fontes

Agente de IA vs. chatbot: como s...

Anthropic - GitHub

Anthropic Claude models - Amaz...

Architecture overview - What is t...

Arquiteturas de IA Agêntica: Padr...

Automação com RAG e Agentes ...

Claude 3.7 Sonnet and Claude Co...

Como criar um agente de IA em P...

Como Criar um Agente de IA: Ch...

Define tools - Claude Platform Do...

Do zero a seu primeiro agente de ...

Extended thinking - Claude Platfo...

Extended thinking - Claude Platfo...

Fundamentals of Building Autono...

How tool use works - Claude Platf...

Introdução - CrewAI Documentat...

Como criar um agente em python usando a API?

Thoughts

Criar um agente de IA em Python utilizando a Messages API do Claude consiste em implementar o ciclo ReAct (Pensar → Agir → Observar → Repetir) 1 2 . Um agente é composto por três pilares fundamentais: o LLM (que toma as decisões), as **ferramentas** (funções Python que executam ações) e o **loop agêntico** (o laço de repetição que mantém o contexto) 1 < 2 3 .

Como Funciona o Contrato de *Tool Use*

Na API do Claude, o modelo nunca executa código diretamente no seu servidor; ele apenas emite um pedido estruturado 4 5 .

- Solicitação:** A aplicação envia a mensagem do usuário junto com o parâmetro `tools` , que descreve as funções disponíveis via JSON Schema 6
- Decisão:** Se o Claude percebe que precisa de uma ferramenta, ele interrompe a geração e responde com `stop_reason: "tool_use"` , fornecendo o nome da função e os argumentos em JSON 4 ...

Faça uma pergunta ou crie algo

32 fontes

Engenheiro de IA, especialista na plataforma Claude

Compartilhado

+ Criar notebook

Fontes

Agente de IA vs. chatbot: como s...

Anthropic - GitHub

Anthropic Claude models - Amaz...

Architecture overview - What is t...

Arquiteturas de IA Agêntica: Padr...

Automação com RAG e Agentes ...

Claude 3.7 Sonnet and Claude Co...

Como criar um agente de IA em P...

Como Criar um Agente de IA: Ch...

Define tools - Claude Platform Do...

Do zero a seu primeiro agente de ...

Extended thinking - Claude Platfo...

Extended thinking - Claude Platfo...

Fundamentals of Building Autono...

How tool use works - Claude Platf...

Execução Local: O seu código Python captura essa solicitação, executa a função localmente e envia o resultado de volta à API dentro de um bloco `tool_result` 6

Repetição: O ciclo se repete até o modelo reunir todas as informações e gerar a resposta final com `stop_reason: "end_turn"` 6

Exemplo Prático em Python (`anthropic` SDK)

O código abaixo demonstra a implementação de um **loop agêntico** manual completo usando o SDK oficial da Anthropic:

```
import json
import anthropic

# 1. Inicializa o cliente da API
client = anthropic.Anthropic()

# 2. Define a função Python local
COTACOES = {"USD": 5.40, "EUR": 5.85}

def obter_cotacao(moeda: str) -> str:
```

Faça uma pergunta ou crie algo

32 fontes

Engenheiro de IA, especialista na plataforma Claude

Compartilhado

+ Criar notebook

Fontes

Agente de IA vs. chatbot: como s...

Anthropic · GitHub

Anthropic Claude models - Amaz...

Architecture overview - What is t...

Arquiteturas de IA Agêntica: Padr...

Automação com RAG e Agentes ...

Claude 3.7 Sonnet and Claude Co...

Como criar um agente de IA em P...

Como Criar um Agente de IA: Ch...

Define tools - Claude Platform Do...

Do zero a seu primeiro agente de ...

Extended thinking - Claude Platfo...

Extended thinking - Claude Platfo...

Fundamentals of Building Autono...

How tool use works - Claude Platf...

Introdução - CrewAI Documentat...

Cuidados Críticos de Implementação

- Condição de Parada (`max_iterations`): É indispensável limitar o número de voltas no loop (`max_iterations`) ¹ - . Sem essa restrição, uma falha na chamada de ferramenta pode fazer o modelo entrar em loop infinito e consumir tokens desnecessariamente ¹² ¹³ .
- Tratamento de Exceções: As funções de ferramentas devem capturar erros e retornar mensagens explicativas em texto no `tool_result` em vez de interromper o script Python ¹⁴ ¹⁵ . O modelo lerá a mensagem de erro e tentará corrigir os argumentos na volta seguinte ¹⁴ ¹⁵ .
- Preservação do Raciocínio Estendido (*Thinking Blocks*): Se você habilitar o raciocínio estendido no modelo (como no Claude 3.7 Sonnet), é obrigatório reenviar todos os blocos `thinking` originais de forma íntegra e não modificada nas chamadas subsequentes do loop agêntico para manter a integridade lógica da sessão ¹⁶ - .

💡 Quer que eu mostre como adicionar **Prompt Caching** para reduzir os custos em loops prolongados, ou prefere ver como conectar esse agente a um servidor MCP (Model Context Protocol) para acessar ferramentas prontas?

Faça uma pergunta ou crie algo

32 fontes

© Gemini Notebook pode cometer erros. Por isso, é bom checar as respostas.

Engenheiro de IA, especialista na plataforma Claude

Compartilhado

+ Criar notebook

Fontes

Agente de IA vs. chatbot: como s...

Anthropic · GitHub

Anthropic Claude models - Amaz...

Architecture overview - What is t...

Arquiteturas de IA Agêntica: Padr...

Automação com RAG e Agentes ...

Claude 3.7 Sonnet and Claude Co...

Como criar um agente de IA em P...

Como Criar um Agente de IA: Ch...

Define tools - Claude Platform Do...

Do zero a seu primeiro agente de ...

Extended thinking - Claude Platfo...

Extended thinking - Claude Platfo...

Fundamentals of Building Autono...

How tool use works - Claude Platf...

Introdução - CrewAI Documentat...

Prompt Caching

Thoughts

O **Prompt Caching** na plataforma Claude permite reutilizar trechos de prompt previamente processados em chamadas de API subsequentes ¹ . Em arquiteturas de agentes de IA — onde o loop agêntico realiza múltiplos turnos enviando o histórico completo de conversas, ferramentas e instruções —, essa funcionalidade reduz os custos de entrada em até 90% e a latência de processamento em até 85% ¹ .

1. Como Funciona a Hierarquia de Caching

O sistema do Claude gera um hash de armazenamento obedecendo a uma ordem estrita e cumulativa de precedência ¹ ² :

- Definições de Ferramentas (`tools`) ¹ ²
- Instruções de Sistema (`system`) ¹ ²
- Histórico de Mensagens (`messages`) ¹ ²

Por ser um hash cumulativo, qualquer alteração em um elemento localizado em uma posição anterior invalida todos os pontos de cache estabelecidos nas etapas seguintes ¹ .

Faça uma pergunta ou crie algo

32 fontes

© Gemini Notebook pode cometer erros. Por isso, é bom checar as respostas.



Fontes

- Agente de IA vs. chatbot: como s...
- Anthropic · GitHub
- Anthropic Claude models - Amaz...
- Architecture overview - What is t...
- Arquiteturas de IA Agêntica: Padr...
- Automação com RAG e Agentes ...
- Claude 3.7 Sonnet and Claude Co...
- Como criar um agente de IA em P...
- Como Criar um Agente de IA: Ch...
- Define tools - Claude Platform Do...
- Do zero a seu primeiro agente de ...
- Extended thinking - Claude Platfo...

2. Estratégias de Ativação

A API disponibiliza duas abordagens para configurar o cache 4 ... :

- **Caching Automático (*Automatic Caching*)**: Adiciona-se o parâmetro `cache_control` no nível superior da requisição de API 4 O sistema aplica e desloca o ponto de interrupção automaticamente para o último bloco elegível à medida que o histórico de conversas do agente se expande 4
- **Breakpoints Explícitos (*Explicit Cache Breakpoints*)**: O parâmetro `{ "type": "ephemeral" }` é inserido diretamente nos blocos de conteúdo desejados (como no final do prompt de sistema, na última ferramenta declarada ou em documentos RAG pesados) 4 É possível definir até 4 pontos de interrupção explícitos por requisição 10 11 .

3. Exemplo Prático em Python (`anthropic` SDK)

Abaixo está um exemplo de como aplicar o **Explicit Breakpoint** no prompt de sistema e nas ferramentas de um agente em Python: